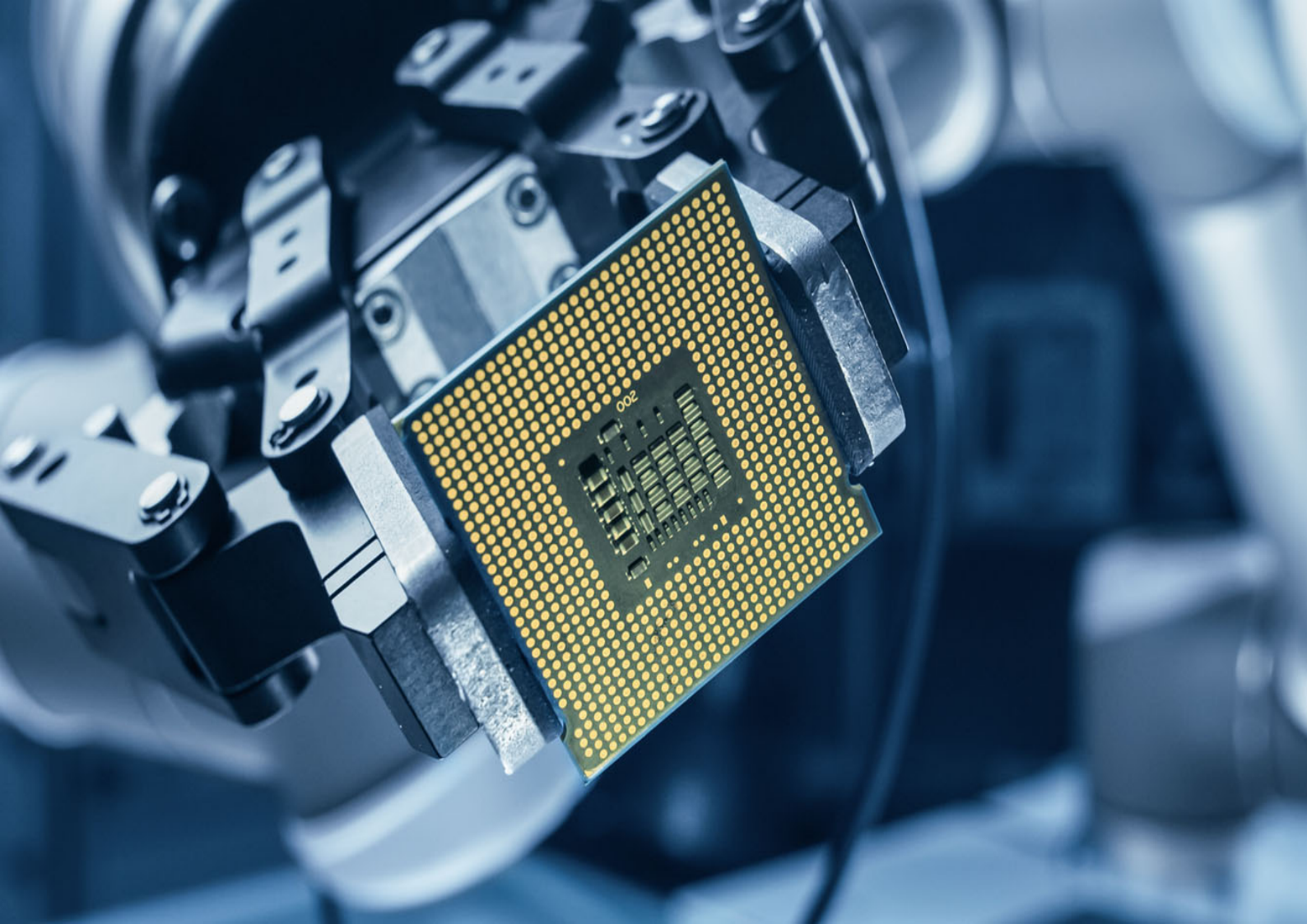




Fuente de corriente Widlar

12 de Agosto del 2023

Bitwise AND

Verificando el estado de un bit

Un uso común de la operación AND bit a bit es verificar si un bit en particular en el operando se encuentra en estado ON u OFF. Por ejemplo sea el siguiente patrón de bits del cual queremos determinar si el cuarto bit se encuentra en ON o en OFF.

8	7	6	5	4	3	2	1
1	0	1	0	1	1	0	1

Ya que queremos determinar el estado del cuarto bit, utilizaremos un segundo operando con el siguiente contenido el cual representa el número 8 en decimal. Esta secuencia de bits es llamada **Bit Mask**.

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
8	7	6	5	4	3	2	1
0	0	0	0	1	0	0	0

El resultado de realizar la operación AND a los bits de la misma posición en el primer y segundo operando obtenemos:

8	7	6	5	4	3	2	1
1	0	0	0	1	0	0	0

El siguiente programa muestra un código de ejemplo para determinar el estado del n-ésimo bit de una determinada cantidad.

```
1  /*
2  +-----+-----+-----+-----+-----+-----+-----+
3  | Position | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 |
4  +-----+-----+-----+-----+-----+-----+-----+
5  | Bits      | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 1 |
6  +-----+-----+-----+-----+-----+-----+-----+ */
7
8  #include<stdio.h>
9  #include<stdlib.h>
10 #include<math.h>
11
12 int value;
13 int position;
14 int posToDec;
15 int bitMask;
16
17 printf("%s", "Enter the quantity in decimal format: ");
18 scanf("%d", &value);
19 printf("%s", "Enter the bit's position you want to supervise: ");
20 scanf("%d", &position);
21
22 posToDec = pow(2, position - 1);
23 bitMask = value & posToDec;
24
25 if (bitMask == 0)
```

```

26     printf("The bit position number %d is OFF", position);
27 else
28     printf("The bit position number %d is ON", position);
29

```

Cambiando el estado de un bit de ON a OFF

Una segunda aplicación de la operación AND bit a bit es para cambiar el estado de un determinado bit, por ejemplo, sea el siguiente patrón de bits:

8	7	6	5	4	3	2	1
0	0	0	0	0	1	1	1

Y queremos cambiar el estado del segundo bit. Para esto, necesitamos un segundo operando que haga el enmascaramiento correspondiente. Esto se logra colocando todos los bits en 1 a excepción de la posición que se requiere cambiar, es decir:

8	7	6	5	4	3	2	1
1	1	1	1	1	1	0	1

Al realizar la operación AND a ambos operandos obtenemos

8	7	6	5	4	3	2	1
0	0	0	0	0	1	0	1

Extrayendo un patrón de bits

RN3	RN2	RN1	RN0	CT	M1	M0	OVF	CRF	FH	FL	L	POL	ME	FC1	FC0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	1	1	1	0	1	1	0	1	0	1	1	1